

深度强化学习与控制

PyTorch 开放性编程实践

说明：本测试旨在考察将 PPT 中的深度强化学习理论 (DQN、DDPG、PPO、Sim2Real) 转化为 PyTorch 计算代码和解决复杂工程问题的能力。请结合理论知识及 PyTorch 编程完成以下开放性题目。

题目 1：DQN 目标网络与损失函数计算

题目背景：DQN 为了解决目标值不稳定和样本相关性问题，引入了目标网络 (Target Network)。目标 Q 值的计算公式为，损失函数采用均方误差。

- 1. PyTorch 实现：**请编写一个 `compute_dqn_loss` 函数。
 - 输入：主网络 Q 值 `q_values`，目标网络 Q 值 `target_q_values`，以及经验回放 batch (`states, actions, rewards, next_states, done`)。
 - 逻辑：计算目标 Q 值（注意处理终止状态 `done=1` 时目标值仅为 `reward`），并计算 MSE Loss。
- 2. 进阶要求：**编写一个 `hard_update(net, target_net)` 函数，实现将主网络参数完全复制给目标网络的功能，

题目 2：DDPG 算法中的“软更新”机制实现

题目背景：DDPG 算法为了保证训练的稳定性，目标网络 (Target Networks) 的参数采用软更新 (Soft Update) 策略，即缓慢跟踪主网络参数：，其中。

- 1. 算法实现：**编写一个通用的 PyTorch 辅助函数 `soft_update(net, target_net, tau)`。
 - 遍历主网络 `net` 和目标网络 `target_net` 的对应参数。
 - 实现上述公式的原地 (in-place) 更新操作。
- 2. 思考与优化：**在 PyTorch 中，如何利用 `model.parameters()` 或 `model.state_dict()` 以及 `data.copy_()` 方法高效实现这一操作，避免不必要的内存开销？

题目 3: PPO 算法的裁剪 (Clipped) 目标函数

题目背景: PPO 算法的核心在于裁剪机制 (Clipping), 用于限制策略更新幅度, 防止性能崩溃。其目标函数为:

其中是新旧策略的概率比率。

1. **PyTorch 实现:** 设计一个 `ppo_loss` 类或函数。

- 输入: 新策略的对数概率 `new_log_probs`, 旧策略的对数概率 `old_log_probs`, 以及优势函数 `advantages`。
- 计算比率。
- 利用 `torch.clamp` 实现裁剪逻辑, 并计算最终的 Policy Loss (注意优化器通常最小化 Loss, 故需取负号)。

题目 4: 基于 Sim2Real 的复杂奖励函数设计

题目背景: PPT 详细列出了机器人动作训练中的奖励函数设计, 包含速度跟踪、能量惩罚、姿态约束等多个分量。合理的奖励设计是 Sim2Real 成功的关键。

1. **类设计:** 假设你有一个四足机器人环境, 请使用 PyTorch 实现一个 `RewardCalculator` 类。

2. **核心实现:** 实现 PPT 表格中列出的至少三种奖励分量:

- **Velocity Tracking (x):** 基于指令速度和实际速度的指数差。
- **Torque Penalty:** 关节扭矩的平方和 (带负权重)。
- **Base Height:** 基座高度与目标高度差的平方 (带负权重)。

3. **功能要求:** 支持通过传入权重字典 (`weights dict`) 对各分量进行加权求和, 输出总奖励。

题目 5: 广义优势估计 (GAE) 的计算

题目背景: 在 PPO 和 Actor-Critic 架构中, 为了平衡偏差和方差, 通常使用广义优势估计 (GAE) 来计算优势函数。公式为:

1. **算法实现:** 编写一个 `calculate_gae` 函数。

- 输入: 奖励序列 `rewards`, 值函数估计序列 `values`, 掩码序列 `done`s, 以及折扣因子和 GAE 参数。
- 逻辑: 倒序遍历轨迹 (从最后一步向前), 递归计算。

2. **输出要求:** 返回计算好的优势值张量 (Tensor), 注意在实现中通过标准化 (normalization) 处理优势值以提升训练稳定性。